

Sanjeevini Multispeciality Hospital Management System: A Role-Based Web Platform for Integrated Healthcare Operations

Karthik P

Department of Data Science Engineering
Ballari Institute of Technology and Management (BITM)
Karnataka, India
Email: *karthikpurma2006@gmail.com*

Abstract—Hospitals manage a large amount of operational and patient-related information including patient registration, appointments, outpatient department (OPD) services, inpatient admissions, electronic medical records, prescriptions, laboratory investigations, pharmacy inventory, billing, and staff activities. Traditional or fragmented approaches to these operations can result in duplicate data, scheduling conflicts, delayed information access, and difficulties in monitoring resources.

This paper presents the design and implementation of the Sanjeevini Multispeciality Hospital Management System, a full-stack web-based healthcare management platform developed using React, Vite, Django REST Framework, and a relational database. The proposed system integrates major hospital operations into a centralized platform and provides role-based access for nine categories of users: Super Administrator, Hospital Administrator, Doctor, Nurse, Receptionist, Pharmacist, Laboratory Staff, Accountant, and Patient.

The system provides modules for patient management, doctor and department management, appointment scheduling, OPD token management, admission and discharge, ward and bed management, electronic medical records, prescriptions, pharmacy, laboratory diagnostics, emergency management, billing and payments, inventory, reports, notifications, and audit logging. Server-side appointment validation is implemented to reduce double-booking conflicts, while bed-status tracking supports monitoring of available, occupied, reserved, and maintenance beds. The system also provides client-side PDF generation for prescriptions and invoices.

The implementation demonstrates how a modular full-stack architecture can integrate different hospital workflows into a unified web application. The system is intended as an academic and engineering prototype and provides a foundation for future extensions such as online payment integration, automated notifications, and telemedicine services.

Index Terms—Hospital Management System, Healthcare Information System, React, Django REST Framework, Role-Based Access Control, Electronic Medical Records, Hospital Automation, Web Application

I. INTRODUCTION

Healthcare organizations handle multiple interconnected activities every day. Patient registration, doctor appointments, medical records, laboratory investigations, pharmacy operations, inpatient admission, billing, and inventory management must be coordinated efficiently. When these processes are

maintained using separate systems or manual records, information may become fragmented and operational tasks may require repeated data entry.

A Hospital Management System (HMS) is a software platform designed to support the digital management of hospital activities. An integrated HMS can provide a centralized interface for managing healthcare workflows while allowing different hospital personnel to access the information required for their responsibilities.

The Sanjeevini Multispeciality Hospital Management System developed in this project is a full-stack web application that combines a React-based user interface with a Django REST Framework backend. The system follows a modular architecture in which hospital activities are separated into functional modules while sharing a common relational data layer.

The major contribution of the project is the integration of multiple hospital operations within a single role-based platform. Instead of treating appointments, medical records, pharmacy, laboratory services, and billing as independent applications, the proposed system connects these processes through shared patient, doctor, staff, and operational records.

The system is designed primarily as an academic and engineering prototype. Its purpose is to demonstrate the application of modern web technologies to hospital workflow management and to provide a foundation that can be extended for real-world environments after additional security, compliance, infrastructure, and clinical validation.

II. PROBLEM STATEMENT

Hospital operations involve a large number of entities and processes. A patient may register with the hospital, book an appointment, receive an OPD token, consult a doctor, undergo laboratory tests, receive a prescription, purchase medicines, and make payments. In the case of inpatient care, the patient may additionally be assigned to a ward and bed, monitored by nursing staff, and eventually discharged.

Fragmented management of these activities creates several challenges:

- Patient information may be distributed across multiple records.
- Manual appointment scheduling can result in conflicting time slots.
- Hospital staff may not have a centralized view of bed availability.
- Handwritten prescriptions can make storage and retrieval difficult.
- Laboratory workflows may be difficult to monitor without status tracking.
- Pharmacy stock levels and medicine expiry dates require continuous monitoring.
- Manual billing can increase calculation and data-entry errors.
- Critical system activities require traceability for administrative monitoring.

Therefore, there is a need for an integrated system that combines these operational workflows into a unified application with controlled access according to user responsibilities.

III. OBJECTIVES

The primary objectives of the proposed system are:

- 1) To develop a centralized web-based platform for hospital management.
- 2) To implement role-based access for different hospital personnel.
- 3) To provide digital patient and doctor management.
- 4) To support appointment scheduling and OPD token management.
- 5) To reduce appointment conflicts using server-side validation.
- 6) To provide electronic medical record management.
- 7) To manage wards, beds, admissions, and discharge operations.
- 8) To provide digital prescription and pharmacy management.
- 9) To manage laboratory test and report workflows.
- 10) To automate billing and payment record management.
- 11) To monitor inventory levels and low-stock conditions.
- 12) To maintain audit logs for important system activities.
- 13) To provide reports and analytics for administrative users.

IV. RELATED WORK

Healthcare Information Systems have evolved from isolated administrative applications into integrated platforms that combine clinical and operational workflows. Electronic Medical Record systems focus primarily on storing and retrieving patient clinical information, while Hospital Information Systems generally extend their functionality to appointments, billing, pharmacy, laboratory services, and administrative operations.

Role-based access control is particularly relevant to healthcare information systems because different categories of users require different levels of access. Doctors may require access to clinical records, while accountants primarily require access to billing information. Similarly, laboratory personnel require

access to diagnostic workflows and pharmacists require access to medicine inventory.

Modern web technologies provide an opportunity to implement these systems using client-server architectures. Component-based frontend frameworks such as React can provide interactive interfaces, while REST-based backend frameworks can expose standardized APIs for data operations.

The proposed system combines these concepts into a single modular architecture. It emphasizes operational integration, role-based access, resource tracking, and workflow automation rather than focusing on one specific clinical specialty.

V. EXISTING AND PROPOSED SYSTEM

A. Existing System

In a fragmented or manual environment, hospital information may be maintained through paper registers, spreadsheets, separate software systems, or independent departmental records. Such an approach can make it difficult to maintain consistency between patient, appointment, billing, laboratory, and pharmacy information.

B. Proposed System

The proposed system provides a centralized web application in which multiple hospital workflows are connected through a common backend and relational database.

TABLE I
EXISTING AND PROPOSED SYSTEM COMPARISON

Existing Approach	Proposed System
Paper or fragmented records	Centralized relational data management
Manual appointment management	Digital appointment scheduling
Manual conflict checking	Server-side double-booking validation
Physical bed checking	Interactive bed-status management
Handwritten prescriptions	Digital prescription management
Manual stock monitoring	Medicine and inventory monitoring
Manual billing calculations	Structured billing and payment records
Limited activity traceability	Audit logging
Department-specific workflows	Integrated hospital modules

VI. SYSTEM ARCHITECTURE

The system follows a layered client-server architecture consisting of a frontend presentation layer, REST API layer, application logic layer, and relational database layer.

The major components are:

- 1) **Frontend Layer:** React and Vite-based web interface.
- 2) **API Layer:** Django REST Framework endpoints.
- 3) **Authentication Layer:** Token-based authentication and role information.
- 4) **Business Logic Layer:** Django views, serializers, validation, and application logic.

5) **Data Layer:** Django ORM with SQLite as the development database.

The frontend communicates with the backend through HTTP requests and JSON responses. Authentication tokens are stored on the client side and included in API requests.

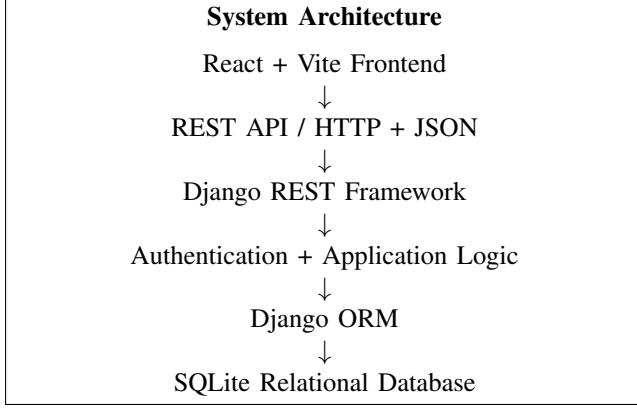


Fig. 1. High-level architecture of the proposed Hospital Management System

VII. TECHNOLOGY STACK

TABLE II
TECHNOLOGY STACK

Layer	Technology
Frontend	React 19
Build Tool	Vite
Backend	Python and Django
API Framework	Django REST Framework
Database	SQLite
Authentication	Token Authentication
Icons	Lucide React
Charts	Chart.js and React Chart.js
PDF Generation	html2pdf.js
Styling	Custom CSS
API Communication	Fetch / HTTP JSON

The frontend package configuration includes React, Vite, Chart.js, React Chart.js, Lucide React, canvas-confetti, and html2pdf.js. The backend uses Django REST Framework, token authentication, CORS support, and the Django ORM.

VIII. ROLE-BASED ACCESS CONTROL

The system defines nine primary user roles:

TABLE III
USER ROLES AND RESPONSIBILITIES

Role	Main Responsibilities
Super Admin	System-wide administration and analytics
Hospital Admin	Staff and department administration
Doctor	Appointments, EMR and prescriptions
Nurse	Patient vitals and ward operations
Receptionist	Registration and OPD operations
Pharmacist	Medicines and pharmacy inventory
Lab Staff	Laboratory tests and reports
Accountant	Billing and payment management
Patient	Appointments, prescriptions and billing information

Role information is represented in the backend user model. The frontend dynamically renders the dashboard according to the authenticated user's role.

For example, the React application maps the following roles to specialized dashboards:

```

SUPER_ADMIN -> SuperAdminDashboard
HOSPITAL_ADMIN -> SuperAdminDashboard
DOCTOR -> DoctorDashboard
NURSE -> NurseDashboard
RECEPTIONIST -> ReceptionistDashboard
PHARMACIST -> PharmacistDashboard
LAB_STAFF -> LabStaffDashboard
ACCOUNTANT -> AccountantDashboard
PATIENT -> PatientDashboard
  
```

IX. FUNCTIONAL MODULES

A. Patient Management

The patient management module stores patient identifiers, personal information, contact details, blood group, medical history, allergies, and emergency contact information. Each patient is assigned a unique patient identifier.

B. Doctor and Department Management

Doctor profiles contain specialization, qualification, experience, department, consultation fee, and availability information. Departments maintain their own identifiers and descriptions and are associated with doctors.

C. Appointment Management

The appointment module connects patients, doctors, departments, dates, and time slots. Appointment states include pending, confirmed, completed, cancelled, and rejected.

A significant feature is double-booking prevention. The serializer validates whether a doctor already has an active appointment for the selected date and time slot.

The conceptual validation condition can be expressed as:

$$Conflict = \begin{cases} 1, & \text{if doctor, date and time slot already exist} \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

If a conflict exists, the requested appointment is rejected.

D. OPD Token Management

The OPD module generates token numbers and token codes for outpatient consultations. It records the patient, doctor, consultation fee, payment state, date, and token status.

E. Ward and Bed Management

The system provides ward and bed management for General, ICU, Emergency, Private, and Semi-Private categories.

Bed states include:

- Available
- Occupied
- Reserved
- Maintenance

The frontend includes an interactive bed grid that visually represents the current bed state.

F. Admission and Discharge

The admission module records admission identifiers, patient information, doctor assignment, bed assignment, admission date, expected discharge date, and discharge information.

Discharge records can include discharge summaries and diagnosis information. The system is designed to release the assigned bed after discharge.

G. Electronic Medical Records

The EMR module stores:

- Patient
- Doctor
- Visit date
- Symptoms
- Diagnosis
- Vital signs
- Treatment plan
- Doctor notes

Vital signs are represented using structured JSON data, allowing the system to store multiple clinical measurements.

H. Prescription Management

Doctors can create digital prescriptions containing patient, doctor, appointment, diagnosis, medicine items, date, and prescription status.

The frontend also supports PDF generation for prescriptions using the `html2pdf.js` library.

I. Pharmacy Management

The pharmacy module manages medicine codes, names, categories, manufacturers, batch numbers, quantities, minimum stock thresholds, prices, expiry dates, and suppliers.

A medicine can be considered low stock when:

$$Quantity \leq MinimumThreshold \quad (2)$$

The system also determines whether a medicine has passed its expiry date.

J. Laboratory Management

The laboratory subsystem manages diagnostic tests and reports. A report follows a workflow containing:

Requested → Sample Collected → Processing → Completed

The workflow can also be cancelled when required.

K. Nursing and Vital Monitoring

Nursing staff can record patient vital measurements including temperature, blood pressure, heart rate, oxygen saturation, respiratory rate, and additional notes.

L. Emergency Management

The emergency module supports emergency case registration with priority levels including Critical, High, Medium, and Low. Emergency cases can be associated with doctors, nurses, and beds.

M. Billing and Payments

The billing module records consultation, room, laboratory, pharmacy, procedure, tax, discount, total, paid amount, and balance amount.

The billing balance can be represented as:

$$Balance = TotalAmount - PaidAmount \quad (3)$$

Payment records support multiple methods such as cash, card, UPI, and online payment.

N. Inventory Management

General hospital inventory is managed separately from pharmacy medicine inventory. Inventory records include item code, name, category, quantity, unit, reorder level, supplier, purchase date, and expiry date.

A low-stock condition occurs when:

$$Quantity \leq ReorderLevel \quad (4)$$

O. Reports and Analytics

The application provides dashboards and reporting functionality for administrative users. Chart.js is used in the frontend to represent selected analytical information.

P. Notifications and Audit Logs

The notification module stores user-specific notifications, messages, types, read status, and timestamps.

Audit logs record user information, user role, action, module, details, IP address, and timestamp. This provides an activity trail for important system operations.

X. DATABASE DESIGN

The backend uses Django models to represent the relational data structure. Major entities include:

- User
- Department
- Doctor
- Patient
- Staff
- Appointment
- OPD Token
- Ward
- Bed
- Admission
- Emergency Case
- EMR Record
- Prescription
- Medicine
- Pharmacy Sale
- Laboratory Test
- Laboratory Report
- Nursing Vitals
- Bill
- Payment
- Inventory Item
- Notification
- Audit Log

Relationships between these entities allow the system to maintain connections between patients, doctors, appointments, admissions, medical records, prescriptions, laboratory reports, billing records, and payments.

For example:

- A department can contain multiple doctors.
- A patient can have multiple appointments.
- A doctor can conduct multiple appointments.
- A patient can have multiple EMR records.
- A patient can receive multiple prescriptions.
- A ward can contain multiple beds.
- A bill can contain multiple payment records.

XI. REST API DESIGN

The Django REST Framework backend exposes REST endpoints for the major system entities.

Examples include:

```
POST /api/auth/login/
POST /api/auth/register/
GET /api/auth/me/

GET/POST /api/departments/
GET/POST /api/doctors/
GET/POST /api/patients/
GET/POST /api/staff/
GET/POST /api/appointments/

GET/POST /api/opd-tokens/
GET/POST /api/wards/
GET/POST /api/beds/
GET/POST /api/admissions/

GET/POST /api/emr/
```

```
GET/POST /api/prescriptions/
GET/POST /api/medicines/

GET/POST /api/lab-tests/
GET/POST /api/lab-reports/

GET/POST /api/bills/
GET/POST /api/payments/

GET/POST /api/inventory/
GET/POST /api/notifications/
GET/POST /api/audit-logs/
```

The frontend communicates with the API through asynchronous HTTP requests. Authentication tokens are attached to protected API requests using the HTTP Authorization header.

XII. SECURITY CONSIDERATIONS

The prototype implements several security-related mechanisms:

- 1) Token-based authentication.
- 2) Password hashing through Django's authentication framework.
- 3) Role information associated with users.
- 4) Authentication-protected API endpoints.
- 5) Server-side appointment conflict validation.
- 6) Audit logging of system activities.
- 7) CSRF middleware provided by Django.
- 8) CORS configuration for frontend-backend communication.

The current implementation is intended for academic development and local demonstration. Production deployment would require additional measures including secure secret management, HTTPS, database hardening, stricter CORS policies, secure production configuration, comprehensive authorization rules, backup policies, monitoring, and compliance with applicable healthcare data regulations.

XIII. SYSTEM WORKFLOW

A typical patient workflow can be represented as follows:

- 1) The patient accesses the public hospital landing page.
- 2) The patient registers or logs into the system.
- 3) An appointment is selected with a doctor and department.
- 4) The backend verifies the selected time slot.
- 5) The patient receives an appointment record or OPD token.
- 6) The doctor accesses the patient and appointment information.
- 7) The doctor creates an EMR record and prescription.
- 8) Laboratory investigations can be requested when required.
- 9) Laboratory staff process tests and update report status.
- 10) Prescribed medicines can be processed by pharmacy staff.
- 11) Billing staff generate and update the patient's bill.
- 12) Payment records are associated with the bill.

For inpatient workflows, admission additionally involves ward and bed assignment. During discharge, the patient's

discharge information is recorded and the associated bed can be returned to an available state.

XIV. IMPLEMENTATION

The frontend is organized into reusable React components, pages, dashboards, modules, contexts, and services.

The major frontend organization is:

```
frontend/
|
+-- src/
|   +-- components/
|   +-- context/
|   +-- pages/
|   | +-- dashboards/
|   | +-- modules/
|   +-- services/
|   +-- App.jsx
|   +-- App.css
|   +-- index.css
```

The backend follows a Django project structure:

```
backend/
|
+-- api/
|   +-- models.py
|   +-- serializers.py
|   +-- views.py
|   +-- urls.py
|
+-- medicare_hms/
|   +-- settings.py
|   +-- urls.py
|   +-- asgi.py
|   +-- wsgi.py
|
+-- manage.py
+-- seed_data.py
```

This modular structure makes it possible to add or modify individual hospital functions without redesigning the entire application.

XV. TESTING AND VERIFICATION

Functional testing was considered for important workflows in the system.

A. Appointment Conflict Test

The appointment serializer checks whether a doctor already has an appointment for the selected date and time slot. An attempt to create another active appointment for the same combination is rejected.

B. Bed Lifecycle Test

The admission workflow associates an inpatient with a bed. During admission, the bed can transition to an occupied state. During discharge, the bed can be released for future allocation.

C. Prescription Export Test

The frontend provides a prescription modal and client-side PDF generation functionality. This allows a digital prescription to be converted into a downloadable document.

D. Billing Test

Billing fields support consultation, room, laboratory, pharmacy, procedure, tax, discount, total, paid amount, and balance calculations.

TABLE IV
FUNCTIONAL TESTING SUMMARY

Test Case	Expected Result	Status
User authentication	Valid users can access dashboards	Pass
Patient registration	Patient record created	Pass
Appointment conflict	Duplicate slot rejected	Pass
OPD token creation	Token generated	Pass
Bed assignment	Bed status updated	Pass
EMR creation	Clinical record stored	Pass
Prescription creation	Prescription stored	Pass
Lab workflow	Status can progress	Pass
Billing	Bill and payment stored	Pass
Inventory threshold	Low stock detected	Pass
Audit logging	Activity recorded	Pass

These tests represent functional verification of the academic prototype and should not be interpreted as clinical validation.

XVI. RESULTS AND DISCUSSION

The implemented prototype demonstrates that multiple hospital workflows can be integrated into a single web-based application.

The role-based dashboard structure provides different interfaces for administrative, clinical, operational, laboratory, pharmacy, financial, and patient users. This reduces the need for every user to interact with all system modules.

The appointment validation mechanism addresses one specific operational problem by preventing multiple active appointments for the same doctor, date, and time slot.

The bed management interface provides a centralized representation of ward resources. Similarly, pharmacy and general inventory modules provide structured information about stock levels and thresholds.

The integration of EMR, prescriptions, laboratory reports, billing, and payments creates relationships between operational records. These relationships demonstrate the benefit of using a centralized relational model rather than isolated data files.

The current implementation is a prototype and therefore several aspects require further development before deployment in a production healthcare environment.

XVII. LIMITATIONS

The current system has several limitations:

- 1) SQLite is primarily used for development and demonstration.
- 2) The current application is designed for local or controlled deployment.
- 3) Production-grade cloud infrastructure is not included.

- 4) External payment gateway integration is not implemented.
- 5) SMS and WhatsApp notifications are not integrated.
- 6) Telemedicine functionality is not currently included.
- 7) Clinical decision support and medical diagnosis automation are outside the project scope.
- 8) Additional security hardening is required for production deployment.
- 9) Healthcare regulatory and compliance requirements require separate evaluation.

XVIII. FUTURE SCOPE

The system can be extended in several directions.

A. Online Payment Integration

Payment gateways such as Razorpay or Stripe could be integrated to support secure online payment processing.

B. Automated Notifications

SMS, email, or WhatsApp services could be integrated for appointment reminders, payment notifications, and laboratory report alerts.

C. Telemedicine

WebRTC-based video consultation could enable remote doctor-patient interaction.

D. Cloud Deployment

The application could be migrated from local SQLite development storage to PostgreSQL or another production-grade relational database and deployed using cloud infrastructure.

E. Advanced Analytics

Future versions could incorporate machine learning and data analytics to identify operational trends such as appointment demand, bed utilization, inventory consumption, and hospital resource requirements. Such analytics should be used for operational decision support and should not replace qualified clinical judgment.

XIX. CONCLUSION

This paper presented the design and implementation of the Sanjeevini Multispeciality Hospital Management System, a full-stack web-based platform for integrating major hospital operational workflows.

The system combines patient management, doctor management, department management, appointment scheduling, OPD token management, admission and discharge, ward and bed management, electronic medical records, prescriptions, pharmacy, laboratory diagnostics, emergency operations, billing, inventory, notifications, reports, and audit logging.

The React and Vite frontend provides an interactive role-based interface, while Django REST Framework provides the backend API and business logic. The relational database model connects the major entities involved in hospital operations.

The implementation demonstrates that a modular web architecture can be used to centralize diverse hospital workflows

and provide different interfaces according to user responsibilities. The prototype provides a foundation for future improvements including cloud deployment, secure online payments, automated notifications, telemedicine, and advanced analytics.

The system is intended as an academic engineering prototype. Additional security, infrastructure, clinical validation, privacy controls, and regulatory compliance would be required before considering deployment in a real healthcare environment.

ACKNOWLEDGMENT

The author would like to acknowledge the Department of Data Science Engineering, Ballari Institute of Technology and Management (BITM), for providing the academic environment and support required for the development of this project.

REFERENCES

- [1] Django Software Foundation, "Django Documentation," <https://docs.djangoproject.com/>
- [2] Encode, "Django REST Framework Documentation," <https://www.django-rest-framework.org/>
- [3] Meta Open Source, "React Documentation," <https://react.dev/>
- [4] Vite, "Vite Documentation," <https://vite.dev/>
- [5] Chart.js, "Chart.js Documentation," <https://www.chartjs.org/>
- [6] SQLite Consortium, "SQLite Documentation," <https://www.sqlite.org/docs.html>
- [7] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [8] W. R. Hersh, "Health Care Information Technology: Progress and Barriers," *JAMA*, vol. 292, no. 18, pp. 2273–2274, 2004.
- [9] U.S. Department of Health and Human Services, "Health Information Privacy," <https://www.hhs.gov/hipaa/>
- [10] K. Purma, "Sanjeevini Multispeciality Hospital Management System," GitHub repository, 2026.